Texte, LLM et systèmes de récupération

Jour 3 : le matin, la théorie du texte et des LLM. L'après-midi, vous construisez un système de question-réponse sur des documents réels, et vous le cassez.

Parcours IAS · 8 heures · 4 h de cours puis 4 h de TD

Le déroulé de la journée

0:00	**Cadrage**	où l'on en est, et ce qui change avec du texte
0:15	**A. Le texte**	représenter des mots par des nombres, et le piège
2:00	Pause	15 minutes
2:15	**B. LLM et RAG**	construire un système, et le casser
4:00	Déjeuner	une heure
4:00	**TD 7 : le texte**	classer des avis, et trouver le piège vous-mêmes
5:45	Pause	15 minutes
6:00	**TD 8 : RAG**	construire, mesurer, saboter
7:55	**Clôture**	le projet portfolio est présenté ici

Un modèle ne lit pas des mots. Il ne manipule que des nombres.

Toute la séance tient dans une question : comment transformer « Le service médical rendu est important » en un vecteur de nombres, sans jeter ce qui compte ? Les réponses successives à cette question sont l'histoire du NLP.

A

Le texte, et ce que sa représentation décide

Transformer des mots en nombres, choisir ce que l'on garde, et découvrir un piège qui est dans les données et pas dans le code.

Ce que le NLP fait déjà à l'hôpital

Extraire les médicaments, les doses et les diagnostics d'un compte rendu : c'est la reconnaissance d'entités nommées, la NER.

Classer un avis, un courrier, un message de patient : c'est ce bloc, et le TD 7 de cet après-midi.

Pseudonymiser retirer les noms et les dates avant de partager un texte. C'est encore la NER, et l'on verra qu'elle ne suffit pas.

Chercher et répondre trouver le bon document et en tirer la réponse : c'est le RAG, au bloc B et au TD 8.

Résumer et traduire condenser un dossier, traduire une notice : ce sont les modèles encodeur-décodeur du bloc B.

Pourquoi le texte résiste

L'ambiguïté « été » est une saison, ou le participe du verbe « être ». Seul le contexte tranche, et l'outil doit savoir le lire.

La morphologie le français conjugue, accorde et élide : pour qui découpe aux espaces, « d'attente » n'est pas « attente ».

La négation et l'ironie « ne présente pas de fièvre » dit l'inverse de « présente de la fièvre », et « super, encore quatre heures d'attente » n'est pas un compliment.

Le domaine SMR, ECG, posologies : le vocabulaire médical est rare dans les textes sur lesquels les outils généralistes ont appris.

Les outils : quatre bibliothèques, quatre usages

NLTK la bibliothèque pédagogique : stemmers, corpus, tokenizers. Bonne pour comprendre, rarement pour produire.

spaCy le pipeline industriel : un modèle par langue, ici `fr_core_news_sm`, qui tokenise, étiquette, analyse et trouve les entités en un seul appel.

scikit-learn TF-IDF, Naive Bayes, logistic regression : le baseline du texte, celui du TD 7.

Hugging Face la bibliothèque `transformers` et des milliers de modèles pré-entraînés, chargés en une ligne. La plus puissante, et la plus facile à mal utiliser.

Le prétraitement, étape par étape

« Les patients hospitalisés ont été traités par des antibiotiques, et leurs douleurs diminuaient rapidement. »

1. tokenization : 16 tokens, dont 2 de ponctuation

Les	patients	hospitalisés	ont	été	traités
par	des	antibiotiques	,	et	leurs
douleurs	diminuaient	rapidement	.		

2. stopwords : 6 mots retirés, en gris, et 8 gardés

La liste de spaCy retire « ont », « par », « et »... et garde « été ».

Et « Le patient ne présente pas de fièvre. » devient « patient présente fièvre » : **la négation était dans la liste.**

3. réduire chaque mot à une forme de base

mot	stemming	lemmatization	POS
patients	patient	patient	NOUN
hospitalisés	hospitalis	hospitaliser	VERB
été	été	être	AUX
traités	trait	traiter	VERB
antibiotiques	antibiot	antibiotique	NOUN
douleurs	douleur	douleur	NOUN
diminuaient	diminu	diminuer	VERB
rapidement	rapid	rapidement	ADV

Le stemming coupe la fin du mot et rend parfois un non-mot (« hospitalis », « antibiot »). La lemmatization rend le mot du dictionnaire, et sait, grâce à la POS, que « été » vient ici du verbe « être » et non de la saison.

16 tokens, 6 stopwords retirés, 8 mots gardés, puis deux façons de réduire chaque mot. Le stemming coupe sans dictionnaire, la lemmatization consulte un dictionnaire et la nature du mot.

Ce que l'on croit, ce que la mesure dit

ON CROIT

Retirer les stopwords est un nettoyage sans risque.

LA MESURE DIT

« Le patient ne présente pas de fièvre. » devient **« patient présente fièvre »**. La liste française de spaCy contient « ne », « pas » et « sans ».

LA BONNE VERSION

Un prétraitement se choisit pour la tâche. Pour classer des sujets, retirer les stopwords aide. Pour lire un compte rendu, il **inverse le sens**. Gardez la négation, ou ne retirez rien et laissez TF-IDF pondérer.

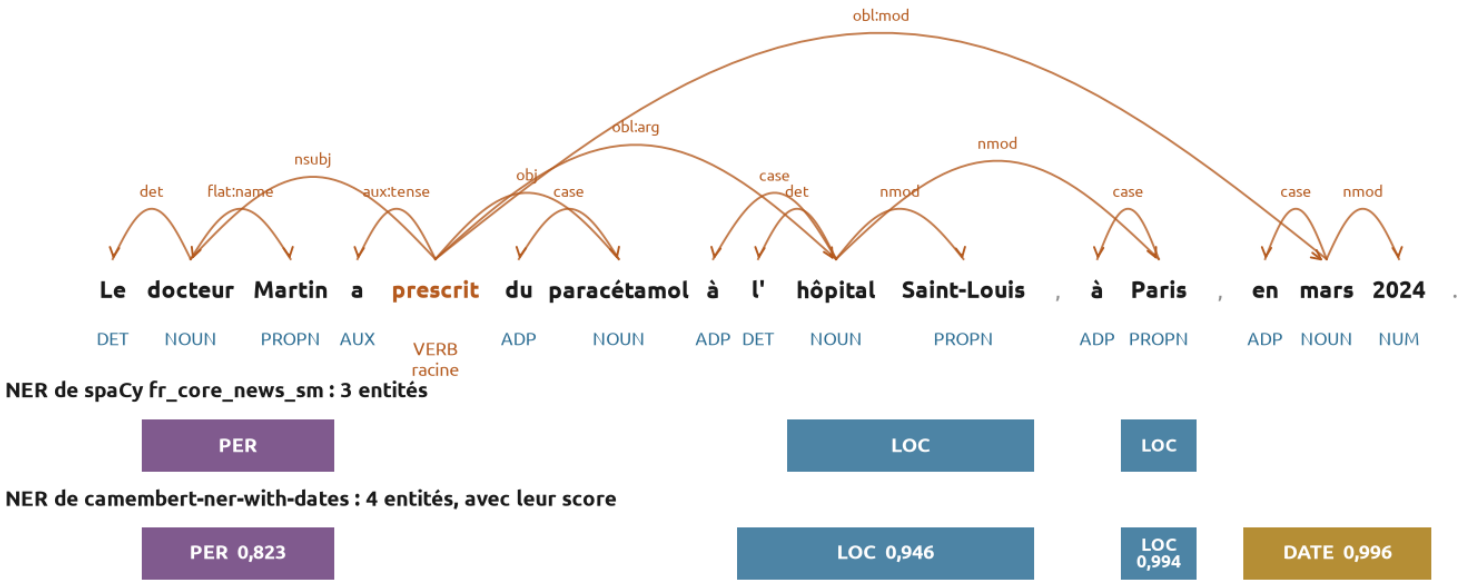
Stemming ou lemmatization : ce que chacun confond

Snowball français de NLTK contre lemmatizer de spaCy, mot par mot.

mots	stemming	lemmatization
traitement, traiter, traitait, traité, traits	trait, pour les cinq	traitement, traiter, trait
douleur, douloureux, douloureuse	douleur, doulour, doulour	douleur, douloureux, douloureux
hôpital, hôpitaux, hospitalier, hospitalisé	hôpital, hôpital, hospitali, hospitalis	hôpital, hôpital, hospitalier, hospitaliser

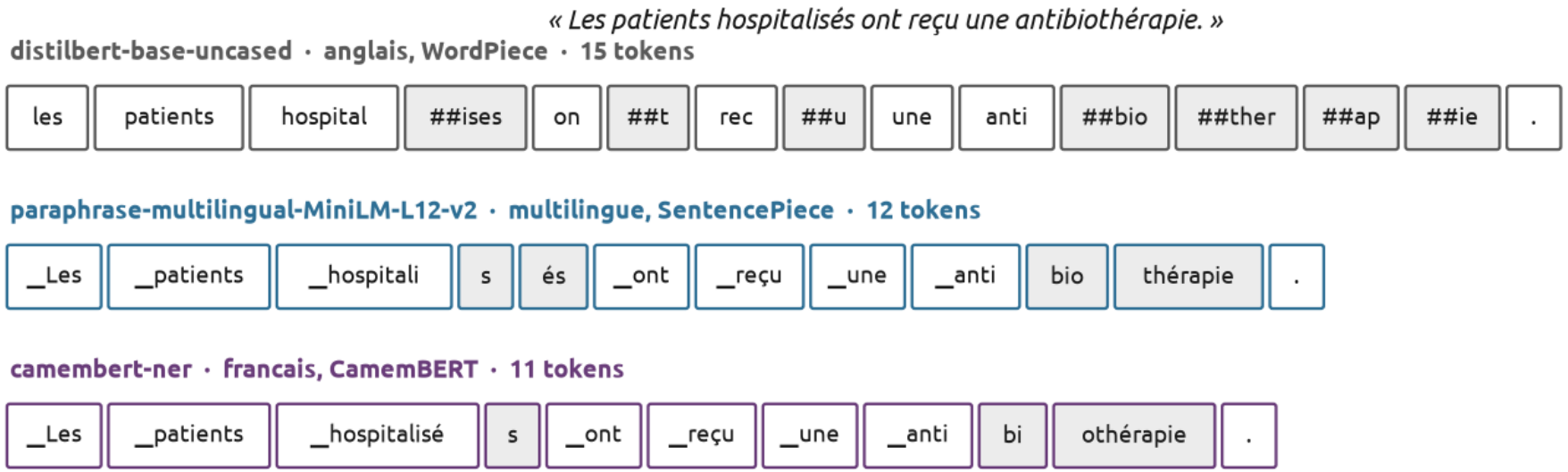
Analyser une phrase : POS, dépendances et entités

Une phrase, analysée : la nature de chaque mot (POS), qui dépend de qui (parsing), et les entités nommées (NER)



spaCy rend la nature de chaque mot et l'arbre des dépendances : « prescrit » est la racine, « docteur » son sujet. Deux NER sur la même phrase trouvent 3 et 4 entités, et seul le second voit la date.

Le tokenizer doit parler la langue



En gris, les morceaux qui continuent un mot : WordPiece les marque par ##, SentencePiece marque le début d'un mot par _. Le tokenizer anglais retire les accents et coupe « ont » en « on ##t » : le modèle ne voit plus les mots français. Le tokenizer est appris avec le modèle, on ne peut pas en changer après coup.

15 tokens pour le tokenizer anglais, 11 pour celui de CamemBERT. L'anglais retire les accents et coupe « ont » en « on ##t » : un modèle anglais ne lit pas le français, il en lit des morceaux.

Trois façons de représenter un texte

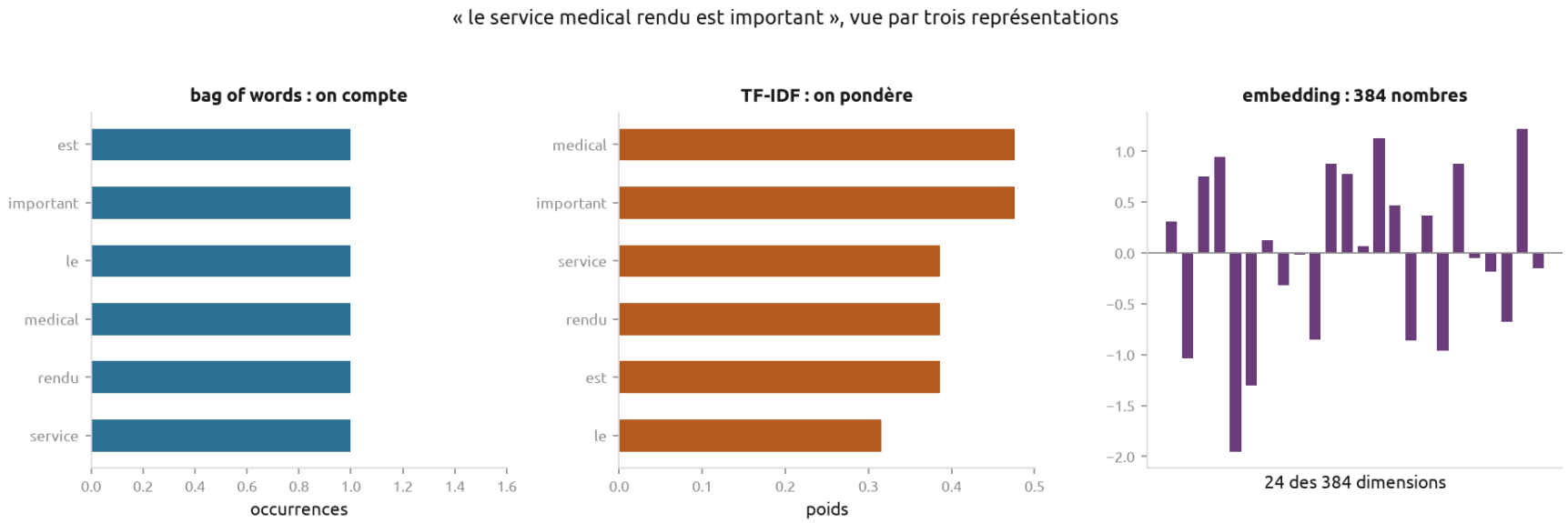
bag of words compter les mots. Simple, rapide, et l'ordre est perdu : « le chat mange la souris » devient identique à « la souris mange le chat ».

TF-IDF compter les mots, mais pondérer : un mot fréquent partout compte moins qu'un mot rare ici. Toujours sans ordre, et toujours très difficile à battre.

embeddings un vecteur appris, où deux textes de sens voisin sont proches même sans mots communs. Coûteux, et pas toujours meilleur.

Le réflexe commencer par TF-IDF. C'est le baseline du texte, et il gagne plus souvent que les cours ne le laissent croire.

La même phrase, vue par trois représentations



Bag of words compte les mots. TF-IDF les pondère, et « medical » et « important » pèsent plus que « le » parce qu'ils sont plus rares. L'embedding rend 384 nombres dont aucun ne veut dire quoi que ce soit tout seul, mais dont l'ensemble porte du sens.

TF-IDF : la formule des manuels n'est pas celle de scikit-learn

scikit-learn : idf = ln((1 + N) / (1 + df)) + 1, puis chaque ligne est normalisée. Trois comptes rendus, N = 3.

mot	documents qui le contiennent	manuel : ln(N / df)	scikit-learn
patient	3 sur 3	0,0000	1,0000
douleur	2 sur 3	0,4055	1,2877
toux	1 sur 3	1,0986	1,6931

La Haute Autorité de Santé publie un avis pour chaque médicament remboursé.

15 315 avis en open data, en français, avec un niveau de service médical rendu pour chacun : important, modéré, faible, ou insuffisant. Peut-on prédire le niveau à partir du texte de l'avis ?

Le corpus, avant toute modélisation

Texte court : médiane 154 caractères. Et le baseline est déjà à 62,9 %.

niveau de SMR	effectif	part
Important	9 631	62,9 %
Insuffisant	2 837	18,5 %
Modéré	1 677	11,0 %
Faible	1 170	7,6 %

Et le classifieur obtient

99,32 %

TF-IDF plus logistic regression, macro F1 de 0,9918.

Contre 62,9 % pour le baseline. C'est un résultat spectaculaire, obtenu en huit dixièmes de seconde, sur du texte clinique réel en français.

Avant de se réjouir, lire un avis

```
Le service medical rendu par BIKTARVY (bictegravir/emtricitabine/
tenofovir alafenamide) 50 mg/200 mg/25 mg, comprime pellicule, est
IMPORTANT uniquement dans le traitement...

# le niveau a predire pour cet avis : Important
```

Le mot du label est dans le texte. Et ce n'est pas un cas isolé : **99,8 % des avis** contiennent le mot de leur propre niveau.

À REFAIRE EN TD 7

Une règle de quatre lignes obtient

99,06 %

Chercher quatre mots. Aucun apprentissage, aucun modèle.

Contre 99,32 % pour le classifieur entraîné, soit 0,26 point d'écart. Le modèle n'a rien appris qu'une expression régulière ne trouvait déjà.

Ce que l'on croit, ce que la mesure dit

ON CROIT

99,3 % d'accuracy sur du texte clinique réel, le modèle a compris.

LA MESURE DIT

Une expression régulière de quatre lignes fait **99,06 %**, parce que 99,8 % des avis contiennent le mot de leur propre niveau. Le modèle a appris à repérer un mot.

LA BONNE VERSION

Un score élevé n'est pas une preuve de compréhension, c'est une question. La bonne question est toujours : **qu'est-ce qui, dans mes features, rend cette tâche facile ?** Et sur du texte, la réponse se trouve en lisant dix exemples.

Et une fois le mot masqué, la vraie tâche apparaît

88,09 %

Macro F1 de 0,8030, contre 0,9918 avant le masquage.

Le macro F1 perd **0,1887**. Et pourtant 88,09 % reste très au-dessus du baseline de 62,9 % : il y a donc bien du signal dans la formulation des avis, au-delà du mot. C'est seulement beaucoup moins spectaculaire.

88,09 % d'accuracy. Et le modèle rate la moitié des avis qui comptent.

Parce qu'une accuracy globale est une moyenne, et que la classe majoritaire pèse 62,9 % de cette moyenne. Il faut regarder classe par classe.

Le détail par classe, après masquage

Le modèle trouve 99,2 % des « Important » et 53,2 % des « Faible ».

niveau de SMR	precision	recall	F1
Important	0,851	0,992	0,916
Insuffisant	0,985	0,822	0,896
Modéré	0,911	0,587	0,714
Faible	0,963	0,532	0,686

Et les trois confusions vont toutes dans le même sens

Tout glisse vers la classe majoritaire.

fréquence	confusion
169 fois	Modéré pris pour Important
129 fois	Faible pris pour Important
119 fois	Insuffisant pris pour Important

Ce que l'on croit, ce que la mesure dit

ON CROIT

L'accuracy a bien baissé, donc elle mesure bien ce qui s'est passé.

LA MESURE DIT

L'accuracy est passée de 0,9932 à 0,8809, soit -0,11. Le macro F1 est passé de 0,9918 à 0,8030, soit **-0,19**, presque le double.

LA BONNE VERSION

Le macro F1 pèse les quatre classes à égalité, donc il chute dès qu'une petite classe est ratée. L'accuracy, elle, est dominée par la classe majoritaire. Sur des classes déséquilibrées, donnez toujours les deux.

Proches par les mots, ou proches par le sens

L'encodeur est celui du TD 8, paraphrase-multilingual-MiniLM-L12-v2.

comparé à « douleur thoracique »	cosinus TF-IDF	cosinus embedding
mal à la poitrine	0,0000	0,6770
douleur abdominale	0,3833	0,6695
fracture du poignet	0,0000	0,5512

TF-IDF et embeddings : quand payer plus cher

TF-IDF gagne quand le vocabulaire est stable et discriminant, ce qui est le cas des textes administratifs et techniques. C'est notre cas.

Les embeddings gagnent quand le sens compte plus que les mots : deux formulations différentes du même contenu, des synonymes, des langues mélangées.

Le coût réel un embedding demande un modèle, de la mémoire, et du temps de calcul à chaque texte. TF-IDF demande une matrice creuse.

La règle mesurez TF-IDF d'abord. Faites prouver à l'embedding qu'il mérite son coût, sur vos données, avec votre métrique.

Le sentiment : une règle, un modèle, et l'ironie

Règle : +1 par mot positif, -1 par mot négatif. Modèle : distilcamembert-base-sentiment, de 1 à 5 étoiles.

phrase	règle, espaces	règle, spaCy	modèle
Le personnel a été remarquable, merci pour tout.	+2	+2	5 étoiles, 0,728
Quatre heures d'attente aux urgences pour un simple pansement.	0	-1	1 étoile, 0,437
Super, encore quatre heures d'attente aux urgences.	+1	0	5 étoiles, 0,746

Zero-shot : classer un commentaire sans un seul exemple

Les six dimensions d'e-Satis, celles du projet du jour 2. Modèle : un mDeBERTa multilingue.

commentaire de patient	1re étiquette	2e étiquette
On m'a laissée sans nouvelles pendant des heures avant de me dire que je pouvais rentrer chez moi.	accueil, 0,549	organisation de la sortie, 0,278
Le plateau était froid et le pain rassis.	repas, 0,766	organisation de la sortie, 0,081

Ce que le bloc A a établi

Le taux de base d'abord, même sur du texte 62,9 % ici, et tout score en dessous est un échec.

Un score trop beau est une question pas une réussite. 99,32 %, et une regex faisait 99,06 %.

Lisez vos données dix exemples lus auraient suffi. Aucune métrique ne l'aurait dit.

TF-IDF est un baseline redoutable commencez par là, et faites prouver à l'embedding qu'il mérite son coût.

Et la tâche reste réelle 88,09 % après masquage contre 62,9 % de base : il y a du signal, il est seulement plus modeste.

Le prétraitement est un choix la liste de stopwords retire la négation, et un tokenizer anglais découpe le français en morceaux.

B

LLM et RAG, vus comme un système

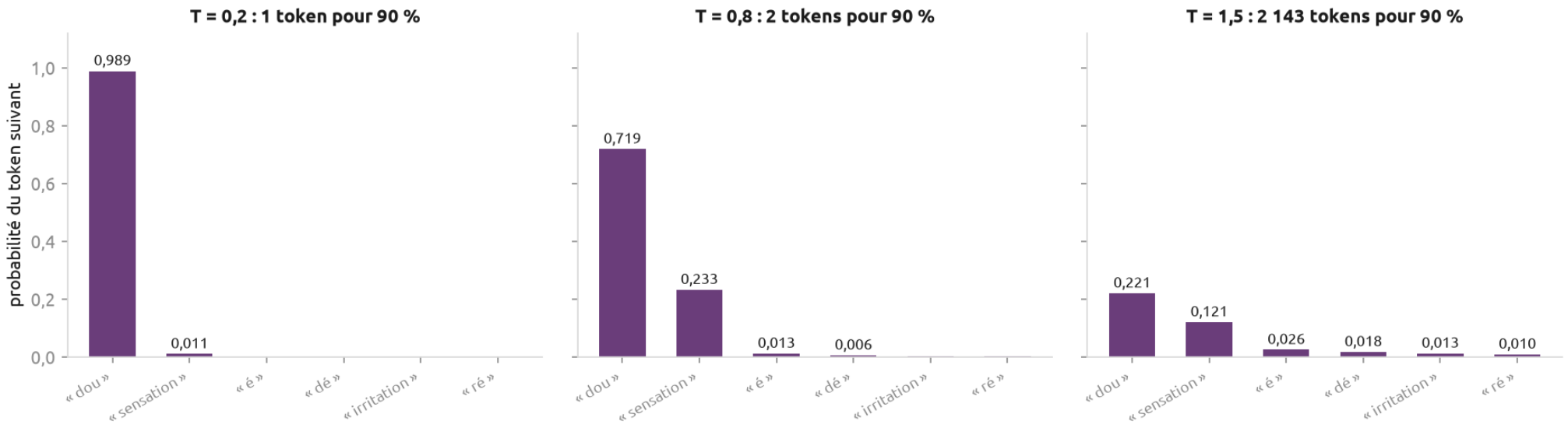
Un modèle de langage n'est pas une base de connaissances. Ce bloc construit le système qui va autour, le mesure, et montre ce qui arrive quand on en retire une pièce.

Un modèle de langage prédit le mot suivant. C'est tout ce qu'il fait.

Il ne consulte rien, ne vérifie rien et ne sait pas ce qu'il ignore. Toutes ses réponses sont produites de la même façon, les justes comme les fausses, et rien dans leur forme ne les distingue.

La température règle la distribution du mot suivant

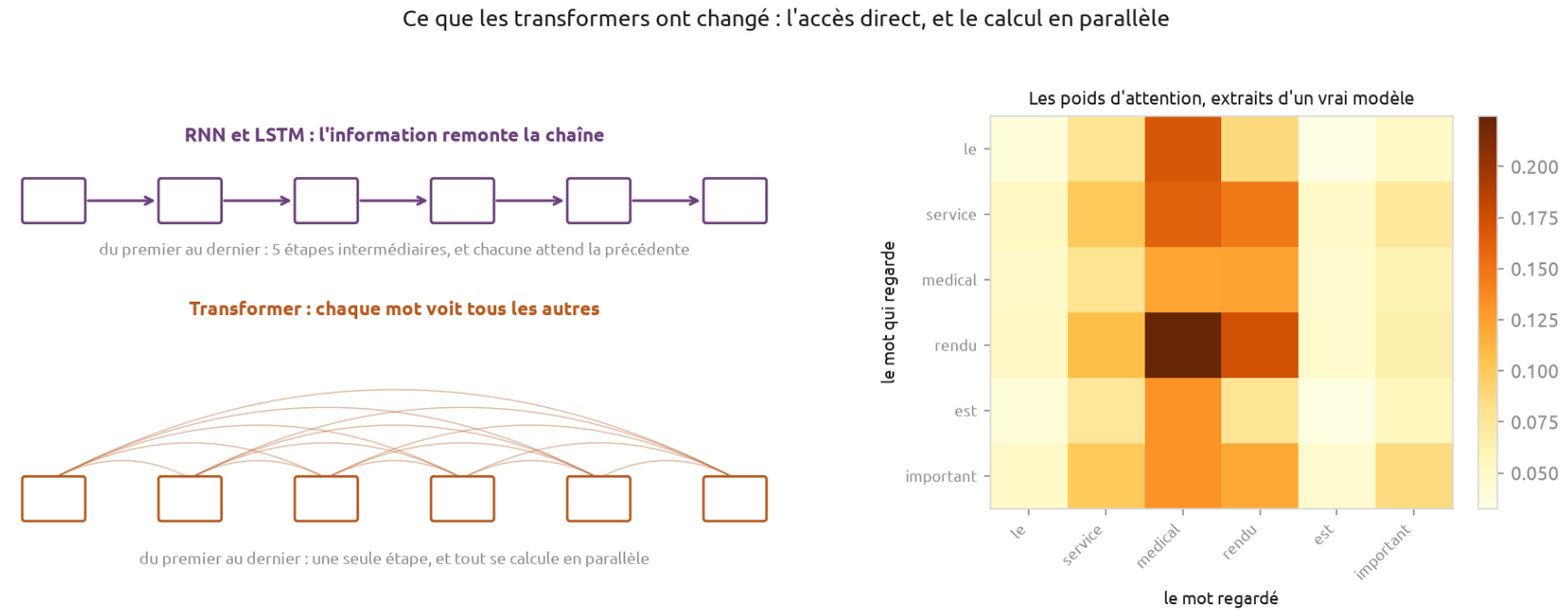
« Après l'opération, le patient a ressenti une vive… », Qwen2.5-1.5B-Instruct, le générateur du TD 8



La température divise les scores avant le softmax : basse, elle écrase la distribution sur le premier choix ; haute, elle l'aplatit.
« dou » n'est pas un mot : c'est le début de « douleur ». Le modèle prédit des tokens, pas des mots.

À T = 0,2, un seul token réunit 90 % de la probabilité ; à T = 1,5, il en faut 2 143. Basse, la température rend le modèle prévisible ; haute, inventif. Dans les deux cas, ce n'est pas la vérité qui change.

Comment il fait : l'attention



À gauche, ce que faisaient les RNN et les LSTM : l'information remonte la chaîne mot après mot. À droite, ce que fait un transformer : chaque mot voit tous les autres en une seule étape, et tout se calcule en parallèle. La matrice de droite est extraite d'un vrai modèle sur cette phrase française : tous les mots regardent « medical » et « rendu ».

Avant les transformers : la récurrence, et ses limites

Le jour 1 a montré le LSTM sur une série temporelle. Un texte est aussi une séquence.

architecture	ce qu'elle apporte	sa limite
RNN	une mémoire : chaque mot met à jour un état caché	le gradient s'évanouit, le début de la phrase s'oublie
LSTM	des portes qui décident quoi garder et quoi oublier	toujours un mot après l'autre, donc lent
GRU	deux portes au lieu de trois, plus léger	la même limite séquentielle
seq2seq	un encodeur lit, un décodeur écrit : la traduction	toute la phrase passe par un seul vecteur
attention	le décodeur regarde tous les mots d'entrée	le transformer ne garde plus qu'elle

L'attention, en une ligne

```
Attention(Q, K, V) = softmax(Q @ K.T / sqrt(d)) @ V

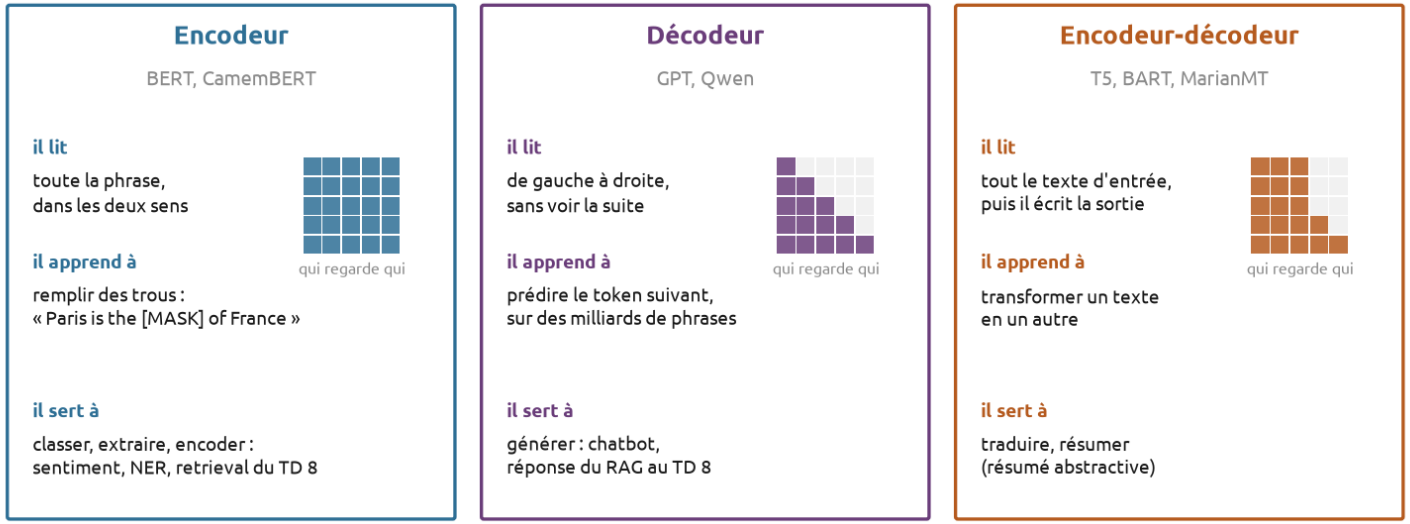
# Q, query : ce que chaque token cherche
# K, key   : ce que chaque token offre
# V, value : ce que chaque token transmet
# sqrt(d)  : sans lui, le softmax sature quand d grandit

# multi-head : h attentions en parallèle, chacune ses Q, K, V
# positional encoding : l'ordre, ajouté au vecteur, que l'attention ignore
```

Une multiplication de matrices, un softmax, une autre multiplication. C'est parce que tout est matriciel que les transformers s'entraînent en parallèle, et c'est parce que l'attention ignore l'ordre qu'il faut le lui rajouter.

Trois familles, trois masques d'attention

Trois familles de transformers : la même attention, des masques différents



Le carré est le masque d'attention : une ligne par token, et une case pleine pour chaque token qu'il a le droit de regarder.

L'encodeur lit toute la phrase et sert à comprendre ; le décodeur ne voit que le passé et sert à générer ; l'encodeur-décodeur lit l'entrée puis écrit la sortie. Les deux modèles du TD 8 sont un encodeur et un décodeur.

Un encodeur apprend en remplissant des trous

distilbert-base-uncased, un modèle anglais : c'est l'exercice de pré-entraînement de BERT, le masked language modeling.

phrase	1re réponse	2e réponse
Paris is the [MASK] of France.	capital, 0,9815	birthplace, 0,0033
The doctor prescribed [MASK] for the pain.	antibiotics, 0,3857	medication, 0,2528

Les tâches, et ce que pipeline() sait faire

Classer sentiment, thème, zero-shot : une étiquette et un score, le bloc A.

Extraire la NER, et le question answering extractif, qui recopie un passage du texte au lieu d'écrire.

Résumer extractive, choisir les phrases importantes ; abstractive, en écrire de nouvelles, au risque d'inventer.

Traduire un encodeur-décodeur par paire de langues, ou un modèle multilingue.

Générer continuer un texte : c'est le décodeur, et c'est la génération du RAG.

Ce que l'on croit, ce que la mesure dit

ON CROIT

Un pipeline de sentiment marche sur n'importe quel texte.

LA MESURE DIT

Sans argument model=, `pipeline("sentiment-analysis")` charge **distilbert-base-uncased-finetuned-sst-2-english** : un modèle anglais, dont le tokenizer coupe « ont » en « on ##t ».

LA BONNE VERSION

Nommez toujours le modèle, et vérifiez sa langue. Ici : distilcamembert-base-sentiment pour le sentiment, camembert-ner pour les entités, un mDeBERTa multilingue pour le zero-shot.

Le fine-tuning : adapter un modèle pré-entraîné

Le principe garder le corps du modèle, qui sait déjà lire, et entraîner une petite tête, qui apprend votre tâche.

Ce qu'il faut des exemples étiquetés, un learning rate faible et peu d'epochs : on ajuste, on ne réapprend pas.

L'outil le `Trainer` de Hugging Face, qui gère les batchs, l'évaluation et les checkpoints.

Le piège oublier d'entraîner. La tête est alors initialisée au hasard, et le modèle répond quand même.

Une tête jamais entraînée répond quand même

0,4870 à 0,4935

P(positif) sur quatre critiques de sens opposés, sans fine-tuning.

Le modèle charge, prédit et ne signale rien. Seul indice : **toutes les probabilités collent à 0,5**, à 0,0130 près. Si vos prédictions valent toutes 0,5, vérifiez que l'entraînement a eu lieu avant d'accuser vos données.

Trois conséquences immédiates, et elles expliquent tout le reste

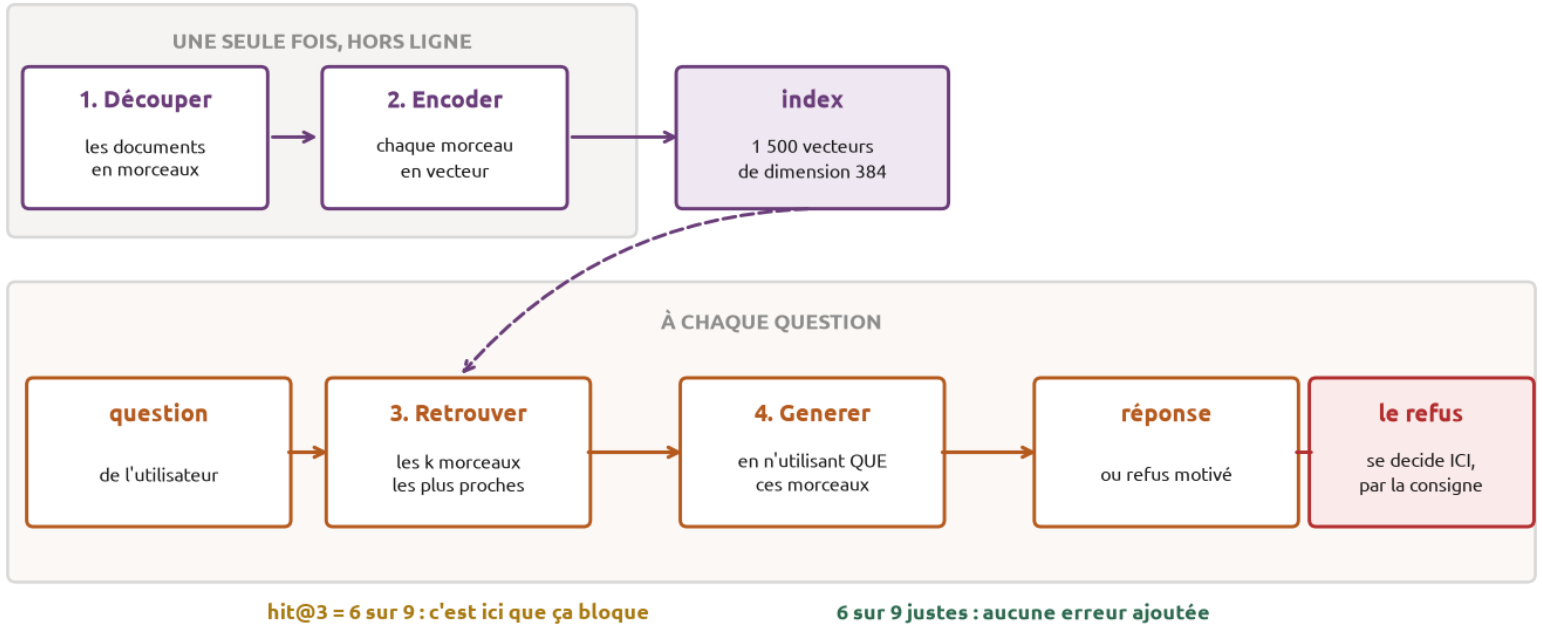
Il invente avec aplomb produire une phrase plausible est exactement son objectif. Une invention bien formée est un succès du point de vue du modèle.

Il ne connaît pas vos documents ses connaissances sont celles de son entraînement, figées, génériques, et sans rapport avec votre organisation.

Il ne sait pas qu'il ne sait pas il n'a aucun mécanisme pour détecter l'absence d'information. Le refus doit venir d'ailleurs.

D'où le RAG retrieval augmented generation : on va chercher les documents pertinents, on les met dans la question, et on interdit de répondre autrement.

Les quatre étapes, et où chaque chose se décide



En haut ce qui se fait une seule fois, hors ligne : découper et encoder. En bas ce qui se fait à chaque question. Les deux chiffres en bas sont mesurés : c'est la recherche qui limite le système, et la génération n'ajoute aucune erreur.

Un RAG tient en quatre étapes

```
# 1. decouper les documents en morceaux
# 2. encoder chaque morceau en vecteur, une fois pour toutes
# 3. a la question : encoder la question, prendre les k plus proches
# 4. generer la réponse EN N UTILISANT QUE ces morceaux
```

Aucune de ces quatre étapes n'entraîne quoi que ce soit. Un RAG n'est pas un modèle, c'est un assemblage, et c'est pour cela qu'on peut le construire en une après-midi.

Le découpage, et les trois réglages qui décident de tout

La taille des morceaux trop grands, le contexte se dilue et le modèle lit surtout du bruit. Trop petits, la phrase utile est coupée en deux et aucun morceau ne répond.

Le nombre k de morceaux repris augmenter k améliore la chance d'avoir la bonne information, et ajoute du bruit autour. Ce n'est pas gratuit.

L'encodeur il doit parler la langue de vos documents. Un encodeur anglophone sur des avis en français est un contresens coûteux.

Et aucun des trois ne s'optimise à l'oeil ils se mesurent sur un jeu de questions, exactement comme un hyperparamètre.

On n'évalue pas un RAG avec des questions auxquelles il sait répondre.

Un jeu d'évaluation qui ne contient que des questions couvertes par le corpus ne mesure jamais le refus. Et le refus est la moitié du problème.

Notre corpus : 1 500 avis de la HAS, un par médicament.

Et douze questions pour l'évaluer : neuf auxquelles le corpus répond, et trois portant sur des médicaments qui n'existent pas. ZYPHORAXINE, MELCORVIDOL et TRANEXIBOR ont été inventés pour l'occasion.

D'abord la retrieval, seule

L'avis utile est-il dans les k premiers documents rendus ?

	résultat
hit@1	5 sur 9
hit@3	6 sur 9
hit@5	6 sur 9

Ce que l'on croit, ce que la mesure dit

ON CROIT

Si le document n'existe pas, la recherche ne rendra rien.

LA MESURE DIT

Sur ZYPHORAXINE, un médicament **qui n'existe pas**, la similarité du meilleur document est de **0,719**, et le système rend un avis sur ZOLADEX. Pour MELCORVIDOL, 0,634. Pour TRANEXIBOR, 0,684.

LA BONNE VERSION

Une recherche vectorielle **classe**, elle ne décide pas. Elle rend toujours les k plus proches, même quand les k plus proches n'ont aucun rapport. Le refus ne peut donc pas venir d'elle.

Puis la génération, avec le contexte

Qwen2.5-1.5B-Instruct, 1,54 milliard de paramètres, sur processeur.

	résultat
réponses justes, sur les 9 questions répondables	6 sur 9
refus corrects, sur les 3 médicaments inventés	3 sur 3

Et les deux chiffres sont les mêmes

6 sur 9, et 6 sur 9

Le hit@3 de la retrieval, et les réponses justes du générateur.

Chaque fois que le bon avis était dans le contexte, le modèle a répondu juste. La génération n'a ajouté **aucune** erreur. Le goulot d'étranglement est la retrieval, et pas le modèle.

Ce que l'on croit, ce que la mesure dit

ON CROIT

Les réponses sont décevantes, il faut un modèle plus gros.

LA MESURE DIT

Les trois échecs sont exactement les trois cas où la retrieval n'a pas trouvé le bon avis. Dans les six autres, un modèle de 1,5 milliard de paramètres a répondu juste.

LA BONNE VERSION

Un modèle plus gros ne peut pas lire un document qu'on ne lui a pas donné. Mesurez la retrieval séparément **avant** de changer de générateur, sinon vous paierez plus cher pour le même résultat.

Maintenant on enlève la retrieval, et on pose les trois mêmes questions.

Les médicaments ZYPHORAXINE, MELCORVIDOL et TRANEXIBOR n'existent toujours pas.

Sans retrieval, sur trois médicaments qui n'existent pas

3 inventions sur 3

Aucun refus. Trois réponses complètes, fluides et fausses.

Et avec la retrieval, sur les mêmes trois questions : trois refus sur trois. La seule différence entre les deux systèmes est une consigne et un contexte.

La même question, deux systèmes

```
Question : Quel est le service medical rendu de ZYPHORAXINE ?

AVEC retrieval :
  Information absente des documents fournis.

SANS retrieval :
  ZYPHORAXINE est un traitement pour les troubles du sommeil,
  notamment l'insomnie et le sommeil bruyant.
```

Le modèle a inventé une **indication thérapeutique complète** pour une molécule qui n'existe pas, en une phrase parfaitement formée et parfaitement plausible. Rien dans sa forme ne la distingue d'une réponse exacte.

Ce qu'il faut en retenir pour construire

Le refus est une décision d'ingénierie il ne vient ni du modèle ni de la recherche. Il vient d'une consigne explicite, et il se teste.

Mesurez les deux moitiés séparément hit@k d'un côté, exactitude de l'autre. Sinon vous ne saurez pas quoi réparer.

Un corpus muet n'est pas une erreur les questions sans réponse doivent faire partie du jeu d'évaluation, sinon vous ne mesurez jamais le refus.

Et le refus est un actif chaque refus est une question que vos documents ne couvrent pas. C'est la liste de ce qu'il faut écrire.

Les pannes classiques d'un RAG, et ce que nos mesures en disent

Chaque panne a sa mesure. Celle qui n'en a pas est celle qu'on ne verra pas.

panne	ce que l'on a mesuré
la recherche rate le bon document	hit@3 de 6 sur 9 : ce sont les trois échecs du système
la recherche ne sait pas dire « rien »	0,719 de similarité pour un médicament inventé
le générateur invente sans contexte	3 inventions sur 3, sans retrieval
le refus manque	il vient de la consigne : 3 refus sur 3 avec elle
le corpus vieillit	rien ne le signale, d'où la clinique système

Une application RAG complète tient en une page

```
# rag_app.py, du dossier NLP_Course : Streamlit, LangChain, FAISS

# 1. le corpus      : documents.txt, six définitions en français
# 2. l'encodeur     : all-MiniLM-L6-v2, anglophone
# 3. l'index        : FAISS IndexFlatL2, une recherche exacte
# 4. la recherche   : k = 1, le seul document le plus proche
# 5. le générateur  : GPT-2, via LangChain, temperature 0.7
# 6. l'interface    : Streamlit, une question, le document, la réponse
```

Un bon squelette, qui affiche le document trouvé : c'est déjà la citation. Et chacun de ses réglages se relit avec ce matin : k=1, c'est le hit@1, **5 sur 9** chez nous ; aucune consigne de refus ; deux modèles anglophones sur du français.

Un chatbot, c'est un RAG avec une mémoire

La mémoire l'historique de la conversation est renvoyé à chaque tour, et il consomme le contexte du modèle.

La reformulation « et pour lui ? » ne se cherche pas tel quel : on réécrit la question avant la retrieval.

Les garde-fous une consigne de refus, la citation des sources, des filtres en entrée et en sortie.

L'évaluation hit@k, exactitude et refus justes, les trois mesures du TD 8, puis ce qu'en disent les utilisateurs.

Ce système est une démonstration d'ingénierie. Ce n'est pas un dispositif médical.

Le corpus a été choisi pour cela : un avis de SMR porte sur le remboursement, pas sur une posologie. Vous allez passer l'après-midi à essayer de casser ce système, et c'est exactement ce qu'il faut faire avec un système que l'on envisage de déployer.

Clinique système : déploieriez-vous ceci ?

1. Qui répond le système se trompe sur un avis et quelqu'un agit dessus. Qui en répond, vous, le service, ou l'éditeur du modèle ?

2. La mise à jour la HAS publie de nouveaux avis chaque mois. Qui ré-encode le corpus, à quelle fréquence, et comment le sait-on ?

3. La dérive silencieuse le système ne tombe pas en panne quand il devient mauvais, il continue de répondre. Comment le détectez-vous ?

4. Le coût 1 500 avis encodés en 9 secondes. Et 150 000 ? Et à chaque question ?

5. L'arrêt que fait le service le jour où le système est indisponible ?

Ce qui manque encore pour que ce soit un système

La citation une réponse doit pointer l'avis dont elle sort, sinon personne ne peut la vérifier et la confiance devient aveugle.

Le journal chaque question, chaque document repris, chaque réponse. Sans cela aucun incident n'est analysable.

La boucle de retour un moyen pour l'utilisateur de signaler une réponse fausse, et quelqu'un qui lit ces signalements.

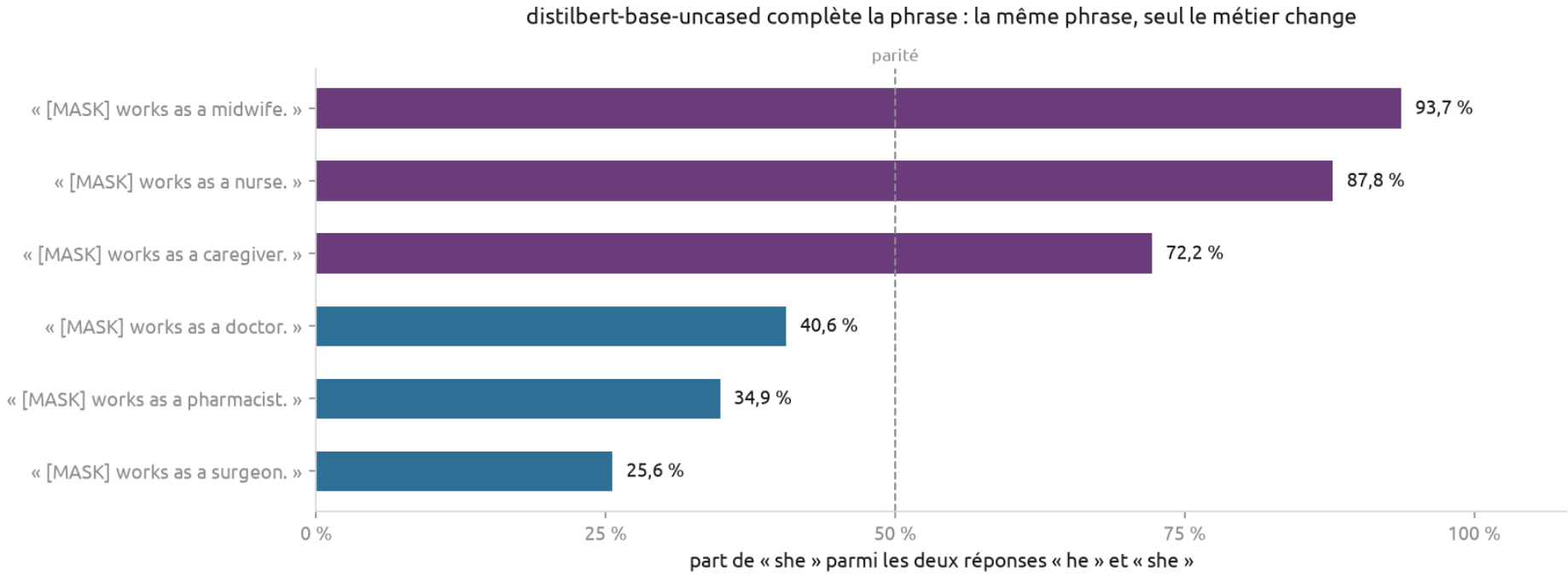
Le jeu d'évaluation vivant douze questions suffisent pour apprendre, pas pour exploiter. Il doit grandir avec les incidents.

Et surtout le journal des refus, qui est la liste mesurée de ce que vos documents ne couvrent pas.

Un modèle apprend aussi ce que ses textes ont de pire.

Personne ne programme un biais. Il arrive avec les données d'entraînement, il ne se lit dans aucune ligne de code, et on ne le trouve qu'en le cherchant.

Un test de substitution : seul le métier change



Personne n'a écrit de règle « nurse = she » : le modèle l'a apprise de ses textes d'entraînement. Rien ne l'annonce, et c'est pourquoi on mesure.

« nurse » : 87,8 % de « she ». « surgeon » : 25,6 %. Le modèle n'a pas de règle, il a des régularités apprises, et elles reproduisent les stéréotypes de ses textes.

D'où viennent les biais, et comment les mesurer

Les données qui écrit, sur quels sujets, dans quelle langue : le corpus n'est pas la population.

L'annotation les étiquettes portent les jugements de ceux qui les posent.

L'objectif un modèle qui maximise l'accuracy sert d'abord la classe majoritaire : le bloc A l'a montré.

Les tests la substitution, comme ici ; WEAT sur les embeddings ; WinoBias et StereoSet pour les modèles de langue.

La règle mesurer la performance par sous-groupe, et pas seulement en moyenne.

Un NER ne suffit pas à pseudonymiser

Texte fictif : une patiente, sa date de naissance, un médecin, un CHU, un numéro de portable.

identifiant	spaCy	camembert-ner-with-dates
Mme Dupont	masqué, PER	masqué, PER
la date de naissance	reste	masqué, DATE
Dr Martin	masqué, MISC	masqué, PER
CHU de Lille	« de Lille » seulement	masqué, LOC
le numéro de portable	reste	reste

Réduire un biais, à trois moments

Avant rééquilibrer et documenter les données, augmenter par substitution contrefactuelle : chaque phrase en « he » reçoit sa jumelle en « she ».

Pendant contraindre l'entraînement, ou choisir un modèle évalué sur des tests de biais publiés.

Après filtrer et corriger les sorties, et surtout garder un humain qui décide.

Toujours publier ce qui a été mesuré, et ce qui ne l'a pas été.

Le cadre : ce que la loi demande déjà

Le RGPD les données de santé sont une catégorie particulière : base légale, minimisation, pseudonymisation, analyse d'impact.

L'AI Act le règlement européen sur l'IA, en vigueur depuis le 1er août 2024, avec des obligations échelonnées selon le niveau de risque.

La transparence dire à l'utilisateur qu'il parle à une machine, et d'où vient la réponse.

La responsabilité un humain signe la décision : c'est la question 1 de la clinique système.

L'après-midi, et la suite

TD 7 classer les avis de la HAS, trouver le piège vous-mêmes, et mesurer ce qu'il valait.

TD 8 construire les quatre étapes du RAG, mesurer la retrieval et la génération séparément, puis saboter.

Le projet portfolio présenté à la clôture, à faire seul après le cours. Il reprend les trois journées, de la licence au pitch.

Ce qui ne change jamais le baseline d'abord, la métrique choisie avant, et la question de savoir d'où vient le score.

Après le cours : votre projet portfolio

Un problème, trois volets prédire et décider, lire et interroger, rendre compte. Les trois journées, dans l'ordre.

Quatre pistes vérifiées médicaments, urgences, qualité des soins, accès aux soins. Ou la vôtre, licence lue.

Onze étapes de la licence au pitch, chacune liée au bloc et au TD qui l'enseignent.

Seul, dix semaines environ quatre heures par semaine, à terminer avant de candidater.

Ce qui vous distinguera pas le score, mais la fuite que vous aurez trouvée et la décision que vous aurez éclairée.